Itinera Key Function Documentation

Frontend

Function Name: App

Location: frontend/src/App.tsx

Input parameters:

None (React component; it does not accept explicit props, relies on context/navigation).

Return value:

A React element tree:

- <UserProvider> wrapping <ItineraryProvider>, wrapping a <NavigationContainer> containing a root NativeStackNavigator with screens:
    - Login
    - SignUp
    - MainTabs
    - NewItinerary
    - ItineraryDetail
    - EditItinerary

Function Overview:

App is the root component of the Expo/React Native app. It:

- Sets up global user state via UserProvider.
- Sets up itinerary state via ItineraryProvider.
- Configures the root navigation stack (login/signup, main tab layout, and detail itinerary screens).
- Disables the default header for all stack screens (headerShown: false) because custom headers are implemented inside individual screens.

Function Name: `signUpWithUsernamePassword`

Location: frontend/src/services/auth.ts

Input parameters:

- username: string
  Username chosen by the user.
- password: string
  Plaintext password to be sent to the backend's /auth/signup endpoint.

Return value:

Promise<AuthResponse | null>

- AuthResponse has shape: { token: string; user_id: number }.
- Returns:
  - AuthResponse on success.
  - null when:
    - Username already exists (HTTP 400 with "Username already exists"), or
    - Other non-validation server errors occur.
  - Throws an error for validation errors (HTTP 422).

Function Overview:

Sends a POST request to POST {BASE_URL}/auth/signup to create a new user. Handles three main scenarios:

1. Success: returns the auth payload containing JWT token and user ID.
2. Username conflict: returns null so the caller can show "Username already taken".
3. Validation error (422): rethrows the Axios error for the caller to handle form-level errors explicitly.

Function Name: `loginWithUsernamePassword`

Location: frontend/src/services/auth.ts

Input parameters:

- username: string
- password: string

Return value:

Promise<AuthResponse | null>

- Returns AuthResponse on success.
- Returns null when the backend responds with:
    - HTTP 400 and { detail: "No such username" }, or
    - any other non-specific failure.

Function Overview:

Performs login against POST {BASE_URL}/auth/login. On valid credentials it returns a token and user ID. On invalid username (specifically “No such username”) or other failures, it returns null so the UI can display a generic “invalid login” message without crashing.

Function Name: `extractPolyline`

Location: frontend/src/pages/Home/Home.tsx

Input parameters:

- it: BackendItinerary
  Backend itinerary object that may include a feature_collection field containing a GeoJSON FeatureCollection.

Return value:

LatLng[][] (array of polyline segments)

- Each inner LatLng[] is one contiguous LineString path extracted from the GeoJSON.
- Only includes valid LineString geometries with at least 2 coordinate pairs.

Function Overview:

Parses a backend-provided GeoJSON FeatureCollection attached to an itinerary, filters out all LineString features, and converts their coordinates from GeoJSON format ([lon, lat]) to React Native Maps format ({ latitude, longitude }). This is used to draw the itinerary route overlays on the MapView in the Home screen.

Function Name: Home

Location: frontend/src/pages/Home/Home.tsx

Input parameters:

None (relies on hooks: useUser, useNavigation, useSafeAreaInsets).

Return value:

A React element tree rendering:

- A full-screen MapView showing:
  - Polylines for each backend itinerary.
  - Markers for POIs of the currently selected itinerary.
- Zoom in/out floating buttons.
- Map-type toggle (standard / satellite).
- A floating “+ New Itinerary” button.
- A bottom sliding card showing details of the selected itinerary.

Function Overview:

Home is the main map screen. It:

- Fetches itineraries from the backend via apiGetItineraries(token) whenever the screen comes into focus.
- Renders each itinerary’s route as a polyline on the map using extractPolyline.
- Allows the user to tap a route to:
  - Select that itinerary,
  - Fit the map camera to its coordinates,
  - Show a bottom sheet card with name, date, time, categories, and cover photo.
- Renders markers for each POI in the selected itinerary with category, average visit time, and cost in the marker tooltip.
- Provides controls to:
  - Zoom in/out programmatically.
  - Toggle between standard and satellite map types.
  - Navigate to NewItinerary to create a new trip.
  - Open ItineraryDetail for more detailed information when the bottom card is tapped.

Function Name: `ItinerariesList`

Location: frontend/src/pages/Home/ItinerariesList.tsx

Input parameters:

None (uses hooks: useNavigation, useUser, useItineraries, useSafeAreaInsets).

Return value:

React element representing:

- A list of itineraries (cards), or
- An empty placeholder when there are none,
- A floating “+ New Itinerary” button,
- A modal confirmation dialog for deleting an itinerary.

Function Overview:

ItinerariesList is the list view counterpart to the map Home screen. It:

- Loads itineraries from the backend via apiGetItineraries(token) on mount and when the screen is re-focused.
- Shows each itinerary in a stylised card (via ItineraryCardBackend).
- Provides a floating “+ New Itinerary” button to navigate to NewItinerary.
- Handles deletion flow:
    - On pressing the trash icon on a card, it shows a confirmation Modal.
    - On confirm:
        - Calls apiDeleteItinerary(id) to delete on the backend.
        - Syncs the local context via deleteItinerary(id.toString()) if successful.
        - Refreshes the list from the backend regardless of result.

Function Name: `ItineraryCardBackend`

Location: frontend/src/pages/Home/ItinerariesList.tsx

Input parameters:

- item: BackendItinerary – the itinerary object from backend.
- onPressDelete: () => void – callback invoked when the trash button is pressed.

Return value:

React element representing a single itinerary card.

Function Overview:

ItineraryCardBackend renders a visual card summarising one itinerary:

- Shows up to 3 cover photos (1 wide image or 1+2 collage layout).
- Displays:
    - Trip name,
    - Budget,
    - Description (if any),
    - Start location or name,
    - Date (e.g. "November 24").
- Navigates to ItineraryDetail with backendId: item.id when the card is pressed.
- Has a floating trash icon in the corner; pressing it triggers onPressDelete to let the parent handle the delete confirmation and API call.

Function Name: `computeTotalTimeLabels`

Location: ItineraryDetail.tsx

Input parameters:

- geoListData: Array<{ opening_time?: string }>
  List rows derived from GeoJSON-based POI points, including optional opening_time.
- pois: BackendItineraryPOI[]
  Raw POI objects from backend, including arrival_time and departure_time.

Return value:

{ rangeLabel: string | null; durationLabel: string | null }

- rangeLabel: e.g. "09:00 - 17:00" (start and end in "HH:MM").
- durationLabel: e.g. "8 hrs" or "7 hrs 30 mins".
- Both null if valid times cannot be computed.

Function Overview:

Computes:

1. The overall time range of the itinerary (start to end).
2. The total duration between those times.

It prefers GeoJSON-derived opening_time values when available; otherwise, it falls back to POI arrival/departure times (sorted by order_index). Used to display a compact, high-level (09:00 - 17:00) 8 hrs label in the itinerary header.

Function Name: `computeRegionFromCoords`

Location: ItineraryDetail.tsx

Input parameters:

- coords: LatLng[] – list of map coordinates to frame.

Return value:

Region | undefined

- Centre latitude/longitude as the midpoint of min/max values.
- latitudeDelta/longitudeDelta sized to include all points with padding.
- undefined if coords is empty.

Function Overview:

Calculates a Region object that fits all given coordinates with some padding, used to drive animateToRegion for the “Fit” control and initial camera framing.

Function Name: `ItineraryDetail`

Location: frontend/src/pages/Home/ItineraryDetail.tsx

Input parameters:

None directly; relies on hooks:

- useRoute – to get params (e.g. backendId, possibly cardId).
- useNavigation – for navigation back and options.
- useItineraries – to look up the client-side card (if provided).
- It also calls apiGetItinerary, apiGetPOIs to fetch backend data.

Return value:

React element tree for the detailed itinerary screen.

Function Overview:

ItineraryDetail shows a **detailed breakdown of a single itinerary**, including:

- Header with:
    - Cover image,
    - Trip name and date,
    - Budget vs actual cost (POIs + travel),
    - Optional total time range and duration.
- A “List / Map” toggle.
- **List view:**
    - Timeline layout of POIs:
        - Start/end markers on the axis.
        - “Travel X mins” labels between stops.
        - For each POI: name, categories, arrival/exit times, stay duration, cost.
- **Map view:**
    - A map with markers for:
        - Start location,
        - GeoJSON points or POIs,
        - Edge labels showing travel time between consecutive POIs.
    - A polyline for:
        - GeoJSON route segments (preferred), or
        - Fallback route built from startCoord + POIs.
    - Controls for:
        - Map type (standard/satellite),
        - Zoom in/out,
        - Fit-to-route.

It also includes an **“Edit details”** modal that lets the user:

- Edit start time, end time, and budget.
- Uses DateTimePicker wheels for time selection.
- On save:
    - Sends PATCH /itineraries/{id} to update the itinerary,
    - Reloads fresh data via apiGetItinerary,
    - Shows success/error alerts accordingly.

Function Name: `DateTimeModal`

Location: NewItinerary.tsx

Input parameters (props object):

- visible: boolean – whether the modal is open.
- initial: Date | null – initial date/time value to show in the picker.
- onClose: () => void – called when user cancels/closes.
- onConfirm: (d: Date) => void – called with the chosen date/time.
- title: string – displayed title (e.g. "Select Start Time").
- mode?: "datetime" | "date" | "time" – determines whether the picker shows date or time (internal implementation currently only uses "date" vs "time" behaviour).

Return value:

React element – modal containing a DateTimePicker and Cancel/Confirm buttons.

Function Overview:

Generic wrapper for @react-native-community/datetimepicker inside a modal overlay. Used for:

- Travel date selection,
- Start time selection,
- End time selection.

It maintains local state for the picker value, initialised each time the modal opens.

Function Name: `pickImage`

Location: NewItinerary.tsx

Input parameters:

- slot: 0 | 1 | 2 – index of image slot to fill; currently only slot 0 is actively used in the compact flow.

Return value:

Promise<void>

Function Overview:

Handles cover-photo selection:

- Requests photo library permission using expo-image-picker.
- If granted, launches image picker for a single image.
- On success, updates the images[slot] URI with the selected asset.
- Swallows errors so UI won't crash if picker fails.

Function Name: `handleCreate`

Location: NewItinerary.tsx

Input parameters:

None (uses multiple state variables and context: name, description, travelDate, startTime, endTime, budgetNumber, startPlace, endPlace, selectedCategories, userId, token).

Return value:

Promise<void>

Function Overview:

Implements the **end-to-end itinerary creation and route generation flow**:

1. Validates that all required fields are satisfied (allRequiredValid). If not, returns early.
2. Sets isCreating to true, which shows a spinner modal with "Planning your itinerary…".
3. Builds backendPayload with:
    - Name, description.
    - date (YYYY-MM-DD from travelDate).
    - cover_photos (first non-empty image).
    - budget with a minimum enforced (e.g. Math.max(budgetNumber, 50)).
    - start_time, end_time converted to "HH:MM:SS".
    - Starting location details (formatted name, coordinates, category).
    - Optional end location details if chosen.
    - Selected categories and user_id.
4. Sends POST {BASE_URL}/itineraries to create the itinerary (with Authorization header if token is present).
5. Reads the created id from the response:
    - If missing/invalid, shows a "Create failed" alert.
6. Calls POST /itineraries/{id}/generate to generate route & POIs:
    - On failure:
        - Attempts to DELETE /itineraries/{id} as a rollback.
        - Shows an "An itinerary could not be generated" alert (or backend detail).
7. On full success:
    - Navigates to ItineraryDetail with backendId: createdId (via navigation.replace).
8. On any error, logs to console and shows general "Create failed" alert.
9. Finally, clears isCreating to hide the spinner.

Function Name: `NewItinerary`

Location: frontend/src/pages/Itinerary/NewItinerary.tsx

Input parameters:

None (uses React Navigation hooks and useUser).

Return value:

React element tree for the create-itinerary flow.

Function Overview:

NewItinerary provides a multi-step trip creation form, optimised for a compact inline “wizard-like” experience:

- Captures:
  - Trip name and description,
  - Travel date,
  - Start location (required), optional end location,
  - Start/end times (with validation that end > start),
  - Budget,
  - Trip categories (from backend via apiGetPoiCategories()),
  - Optional cover photo from device.
- Has two UX modes:
  - Compact inline: main mode where all fields are on a single scroll, with a “Review/Edit” toggle in the header.
  - Step-based (non-compact) flow that’s largely preserved but not used when compact is true.
- Provides a Review screen summarising all selections before creation.
- Uses full-screen modals for:
  - Date/time picking (via DateTimeModal).
  - Autocomplete start/end location selection (LocationAutocomplete).
  - Category selection.
- Shows a blocking spinner modal while the itinerary is being created and route is generated, giving clear feedback to the user.

File: frontend/src/types/expo-location.d.ts

This declares the shape of functions from the expo-location module.

Declared Function: `requestForegroundPermissionsAsync`

- **Input parameters:** none.
- **Return value:** Promise<PermissionResponse> ({ status: PermissionStatus }).
- **Overview:** Asks the OS for location permission when the app is in the foreground.

Declared Function: `getCurrentPositionAsync`

- **Input parameters:** optional { accuracy?: number }.
- **Return value:** Promise<Position> where Position.coords includes latitude and longitude.
- **Overview:** Retrieves the device's current location.

Declared Function: `getLastKnownPositionAsync`

- **Input parameters:** none.
- **Return value:** Promise<Position | null>.
- **Overview:** Returns the most recent known device location, if any.

File: frontend/src/types/react-native-maps.d.ts

Declares types for react-native-maps.

Declared Class: `MapView`

Methods:

- fitToCoordinates(coords: LatLng[], options?: FitToOptions): void
- animateToRegion(region: Region, duration?: number): void

Overview:

Defines how the MapView ref can be used in your components (e.g. in Home and ItineraryDetail) to zoom/fit to specific coordinates.

Declared Components:

- Marker
- Polyline

Each with their typed props (coordinates, titles, stroke colours, etc.). Again, implementations are from the library; these files just provide TypeScript typing.

Backend

Function Name: `get_route`
Location: application/ors_service.py

Input parameters:

`pois`

- A list of POI objects representing the planned itinerary stops in order.
- Each POI dictionary is expected to contain id, name, lat, lon, category
- These provide the metadata to build Point features and route legs

`mode_per_leg`

- A list specifying the travel model for each leg (e.g “driving-car”, “foot-walking”)

`distance_lookup`

- A dictionary mapping a `(from_id, to_id)` tuple → distance in metres.
- Provides travel distances for each leg

`duration_lookup`

- A dictionary mapping a `(from_id, to_id)` tuple → duration in minutes.
- Provides travel times for each leg

Return Value:

- A dictionary in standard GeoJSON format:
- Point Features: name, category, order index and map coordinates
- Route Leg LineStrings: real geometry from ORS or straight line if ORS failed
- Leg metadata: each leg contains mode, distance, duration, from POI name, to POI name

```
{
  "type": "FeatureCollection",
  "features": [
      { ... Point feature ... },
      { ... LineString travel leg ... },
      ...
  ]
}
```

Function Overview:
Generates a complete route representation for an ordered list of POIs. It attempts to retrieve real-world routing geometry using the OpenRouteService (ORS) Directions API. If the API is unavailable or returns insufficient data, the function automatically switches to a fallback mode using straight-line geometry.

The function returns a GeoJSON FeatureCollection which is used for rendering itinerary routes in the frontend.

Function Name: `create_poi_relationships`
Location: application/poi_relationship_service.py

Input parameters:
`existing_pois`

- A list of POIs already stored in the system

`mode`

- The transportation mode for ORS e.g “driving-car”, “foot-walking”

`new_start` (optional)

- Represents a newly added starting POI
- If provided, the function only computes distances/duration involving this POI

`new_end` (optional)

- Represents a newly added starting POI
- If provided, the function only computes distances/duration involving this POI

Return Value:
None.

Function Overview:
Computes and stores pairwise distance and duration relationships between POIs using the OpenRouteService (ORS) Matrix API.

It supports two operation modes:

1. **Full Matrix Mode**
   - Computes a complete $N \times N$ matrix of distances/durations for all existing POIs.
   - Used when the POI dataset is initially populated.

2. **Incremental Update Mode**
   - Runs only when `new_start` and/or `new_end` POIs are provided.
   - Instead of recomputing the entire matrix, the function calculates only the edges involving newly added POIs.
   - This reduces API calls and improves computation efficiency.

Function Name: `create_itinerary`
Location: application/itinerary_service.py

Input parameters:
`Itinerary_request: ItineraryCreate`

- A Pydantic model containing all user-specified itinerary constraints:
  - name
  - description
  - date
  - cover_photos
  - cudget
  - start_time, end_time
  - start_lat, start_long, start_name, start_cat
  - end_lat, end_long, end_name, end_cat
  - categories
  - user_id

Return Value:
`ItineraryResponse`

- A Pydantic response model containing the created itinerary metadata stored in the database

Function Overview:
Creates a new itinerary record based on user-specified constraints such as start location, end location, budget, date, and available time window.

This function does not generate an optimised route. Instead, it:

1. Validates the request parameters.
2. Ensures the user exists.
3. Creates user-specified start and end POIs (if needed).
4. Updates POI-to-POI relationship tables, including new distances and durations using the POIRelationshipService.
5. Stores the itinerary metadata in the database.

This prepares the system for route generation, which is handled by a separate optimisation process.

Function Name: `_prepare_itinerary_data`
Location: application/itinerary_service.py

Input parameters:
`itinerary: ItineraryResponse`

- A Pydantic model representing a stored itinerary

Return Value:
A dictionary containing all optimisation inputs including:

- all_default_pois
- ending_poi
- is_round_trip
- all_categories
- poi_categories
- poi_ids
- utility_scores
- travel_distances
- travel_costs
- visit_costs
- taxi_cost_per_metre
- travel_times
- opening_times
- closing_times
- visit_times
- time_budget

Function Overview:
Constructs and returns all precomputed data structures required by the optimisation model.

Function Name: `_build_model`
Location: application/itinerary_service.py

Input parameters:

`data`

- A precomputed dataset containing all relevant POI properties needed for optimisation, produced by `_prepare_itinerary_data`.

`return_poi_id`

- Id of the final POI
- For round trips: ID of the starting POI
- For straight trips: user-defined ending POI

`poi_ids_with_return`

- List of all POI IDs included in the optimisation

Return Value:

`m`

- Base Gurobi model

`y`

- Binary variable indicating whether POI is selected

`z`

- Binary variables representing directed edges between POIs

`p`

- Continuous variables indicating visit order/position of each POI

`arrival_time`

- Continuous variables representing arrival times at each POI (minutes)

Function Overview:
Constructs and configures a Gurobi optimisation model used to generate an optimal itinerary for a user.
It creates all decision variables (POI selection, route edges, visit order, and arrival times), defines the objective function (maximising total utility score), and adds foundational routing constraints (start location first, end location last, valid entry/exit structure).
The resulting model is later extended with additional constraints (time windows, budget, category constraints, etc.) before being solved.

Function Name: `_add_constraints`
Location: application/itinerary_service.py

Input parameters:

`m`
- Base Gurobi model

`y`
- Binary variable indicating whether POI is selected

`z`
- Binary variables representing directed edges between POIs

`p`
- Continuous variables indicating visit order/position of each POI

`arrival_time`
- Continuous variables representing arrival times at each POI (minutes)

`data`
- A precomputed dataset containing all relevant POI properties needed for optimisation, produced by `_prepare_itinerary_data`.

`itinerary`
- A Pydantic model representing a stored itinerary

`return_poi_id`
- Id of the final POI

`poi_ids_with_return`
- List of all POI IDs included in the optimisation

Return Value:
None.

Function Overview:
Adds all core constraints to the Gurobi optimisation model created in `_build_model`.
These constraints encode:
- Cost budget limits
- Time budget limits
- Arrival-time feasibility (opening/closing hours)
- Sequence and travel-time consistency
- Category-based preferences
- Flow constraints (valid path structure)
- Subtour elimination (MTZ constraints)

Together, they ensure that the model produces a valid, time-feasible, cost-feasible, category-compliant itinerary that respects all routing rules.

The function does not return a value; instead, it modifies the model in place.

Function Name: `_solve_and_extract`
Location: application/itinerary_service.py

Input parameters:

`m`

- Base Gurobi model

`y`

- Binary variable indicating whether POI is selected

`p`

- Continuous variables indicating visit order/position of each POI

`arrival_time`

- Continuous variables representing arrival times at each POI (minutes)

`data`

- A precomputed dataset containing all relevant POI properties needed for optimisation, produced by `_prepare_itinerary_data`.

`return_poi_id`

- Id of the final POI
- For round trips: ID of the starting POI
- For straight trips: user-defined ending POI

`poi_ids_with_return`

- List of all POI IDs included in the optimisation

Return Value:

`ItineraryResponse`

- The final, generated itinerary with updates statistics including:
- Ordered POI sequence with arrival/departure times
- Travel times and distances between POIs
- Total time, cost, distance, and utility score
- Route geometry as a GeoJSON FeatureCollection

Function Overview:

Solves the Gurobi optimisation model for the itinerary, handles infeasibility or errors, and extracts the solution into a detailed itinerary format.

It performs the following steps:

1. Runs the Gurobi optimizer and monitors solve time.
2. Checks for infeasibility and reports clear error messages based on violated constraints.
3. Extracts selected POIs and their positions in the optimal route.
4. Computes arrival/departure times, travel distances, travel costs, total time, total score, and other summary statistics.
5. Retrieves detailed route geometry from OpenRouteService.
6. Updates the database with computed itinerary statistics.
7. Returns the fully generated itinerary in the format expected by the service.

Function Name: `_generate_itinerary`
Location: application/itinerary_service.py
Input parameters:
`itinerary_id`

- Unique identifier of itinerary to be generated

Return Value:
`ItineraryResponse`.

- The final, generated itinerary with updates statistics including:
- Ordered POI sequence with arrival/departure times
- Travel times and distances between POIs
- Total time, cost, distance, and utility score
- Route geometry as a GeoJSON FeatureCollection

Function Overview:
Orchestrates the end-to-end itinerary generation flow. It uses precomputed POI data, builds and solves the optimisation model, applies constraints, and extracts the final itinerary.

It performs the following steps:

1. Retrieve the itinerary request by ID.
2. Prepare POI data and metadata using `_prepare_itinerary_data`.
3. For round trips, add a synthetic return node to model returning to the starting location.
4. Build the Gurobi optimization model with `_build_model`.
5. Add all necessary constraints using `_add_constraints`.
6. Solve the model and extract results via `_solve_and_extract`.
7. Return the fully generated itinerary in a format suitable for storage and downstream usage.

Function Name: `_update_itinerary`
Location: application/itinerary_service.py

Input parameters:
`itinerary_id`
- Unique identifier of itinerary to be updated

`update_data`
- Object containing the fields to update (start time, end time, budget)

Return Value:
`ItineraryResponse`
- The final, generated itinerary reflecting the updated fields and route including:
- Ordered POI sequence with arrival/departure times
- Travel times and distances between POIs
- Total time, cost, distance, and utility score
- Route geometry as a GeoJSON FeatureCollection

Function Overview:
Updates an existing itinerary's budget, start time, and or end time based on user input. It performs validation on the updated fields, applies the changes in the database, and then triggers a full itinerary regeneration to reflect the updated constraints.

Function Name: `delete_itinerary`
Location: application/itinerary_service.py

Input parameters:
`itinerary_id`
- Unique identifier of itinerary to be deleted

Return Value:
True on deletion success, else an error is raised.

Function Overview:
Permanently removes the itinerary record from the database.

Function Name: `create_poi`
Location: application/poi_service.py

Input parameters:
`poi_dict`

- A dictionary containing the attributes required to create a POI including:
    - name
    - longitude
    - latitude
    - opening_time
    - closing_time
    - category
    - intrinsic score
    - avg_visit_time
    - visit_cost
    - is_user_added

Return Value:
`POIResponse`
- A Pydantic response model containing the created POI metadata stored in the database

`None`
- When a POI already exists at the given coordinates

Function Overview:
Attempts to create a new POI in the database.
Before creating, the service checks whether a POI already exists in the database with the same longitude and latitude.
If a matching POI exists, no new POI is created and the function returns None.
Else, the POI is created and the newly created POI is returned.

Function Name: `get_poi_categories`
Location: application/poi_service.py

Input parameters:
None.

Return Value:
`List[str]`
- A list of unique category names extracted from all default POIs.

Function Overview:
Retrieves a unique list of all POI categories that come from the default POIs in the dataset. User added POIs i.e. start and end location POIs, are intentionally excluded to ensure the user cannot select categories of interest from them.